Lecture 25: Integer Optimization

Zizhuo Wang

School of Data Science (SDS)
The Chinese University of Hong Kong, Shenzhen

May 6, 2023

Branch-and-Bound Method

High-level idea:

- Branching: Divide the feasible region into smaller ones, solve each of them and combine them to find the optimal solution
- Bounding: Use bounds (LP optimal value and feasible solutions obtained) to reduce the number of branches we need to consider

Branch-and-Bound Method

Branching Procedures:

1. Solve the LP relaxation
   - If the optimal solution is integral, then it is optimal to IP
   - Otherwise go to step 2
2. If the optimal solution to the LP relaxation is $\mathbf{x}^*$ and x_i^* is fractional, then branch the problem into the following two:
   - 2.1 One with an added constraint that $x_i \leq \lfloor x_i^* \rfloor$
   - 2.2 One with an added constraint that $x_i \geq \lceil x_i^* \rceil$
3. For each of the two problems, use the same method to solve them, and get optimal solution $\mathbf{x}_1^*$ and $\mathbf{x}_2^*$ with optimal value v_1^* and v_2^*
   - Compare and get the optimal solution

Branch-and-Bound Method

Bounding procedures (for maximization):

- Any LP relaxation solution can provide an upper bound for each node in the branching process
- Any feasible solution to the IP can provide a lower bound for the entire problem
- We call the best integer solution obtained for the problem the "incumbent solution"

When at a certain node, the optimal value of the LP relaxation of this branch is even less than the current lower bound (the objective value of the incumbent solution). Then we should abandon this branch (also called prune or fathom that branch)

- No better solution can be obtained from exploring this branch

Bounding is very important for branch-and-bound, it is the key to make it efficient (and practical)

Order of Computing the Branches

When we do branch-and-bound, we may have two choices to do at each step

- In the example in the last lecture, if we compute (S2) first, then we get a non-integral solution and thus two branches.
- Suppose (S1) also has two branches (if there is a fractional solution), then we need to decide to go deep in one of these branches or try all the branches in the same level

Basically, we need to decide if we want to *go deep* into one branch first or *go wide* to solve all problems on a given level first

Deep or Wide?

In branch-and-bound algorithm, the best approach is to *go deep* into the tree, not to *go wide*

- Most integral solutions lie deep in the tree; it's good to have integral feasible solutions early, so we can use it in the *bounding* procedure
- It's also memory-efficient, since each LP is obtained from its parent by merely adding one constraint
- It's also easier to code (recursion)

Selecting the Branch

One of the key to the effectiveness of the branch-and-bound method is in how one selects the branch to compute next

- If one always chooses the "correct" branch, then one is likely to solve the problem very fast
- Unfortunately, there is no work-for-all theory about which branch to select next (when both are not pruned according to the bounding procedure)
- One common heuristic is to first consider the branch with better LP relaxation value (i.e., smaller LP relaxation optimal value if minimizing, and larger LP optimal value if maximizing)
- The selection is often more of a "heuristic" or "experience" than theory — many of the tricks are not openly published (because it affects the speed of the solver)
- Recently, there has been a growing interest in using machine learning methods to determine which branch one should select considering various factors — some moderate success

Complexity of Branch-and-Bound

Branch-and-bound is essentially an enumeration method.

- In the worst case, branch-and-bound may need to go through each feasible integer solutions in the region, which is exponential in the problem size
- Remember there is no polynomial-time algorithm for IP

However, branch-and-bound does enumeration in a smart way and typically it only needs to visit a tiny fraction of all solutions

- Much more efficient than explicitly enumerating the solution
- It is one of the most useful practical methods

Branch-and-Bound for Binary Problem

Branch-and-bound can also be used to solve binary linear programs

Consider the following knapsack problem:

$$\begin{aligned} \text{maximize} \quad & 8x_1 + 11x_2 + 6x_3 + 4x_4 \\ \text{subject to} \quad & 5x_1 + 7x_2 + 4x_3 + 3x_4 \leq 14 \\ & x_j \in \{0, 1\}, \quad j = 1, ..., 4. \end{aligned}$$

Example Continued

The LP relaxation for this IP is:

$$\begin{aligned} \text{maximize} \quad & 8x_1 + 11x_2 + 6x_3 + 4x_4 \\ \text{subject to} \quad & 5x_1 + 7x_2 + 4x_3 + 3x_4 \leq 14 \\ & 0 \leq x_j \leq 1, \quad j = 1, ..., 4. \end{aligned}$$

For this type of problem (one constraint linear optimization), the optimal solution is always to give the maximum value based on the value-to-weight ratio until it becomes infeasible.

In this case, the optimal solution is $x_1 = 1$, $x_2 = 1$, $x_3 = 0.5$, $x_4 = 0$. The optimal value is 22.

- We need to do branching for x_3
- Consider two subproblems, one with $x_3 = 1$ (called (S1)), the other with $x_3 = 0$ (called (S2))

Example Continued

Solve the LP relaxation (S1), we get the optimal solution is $x_1 = 1$, $x_2 = 0.714$, $x_3 = 1$ and $x_4 = 0$. And the optimal value of the LP relaxation is 21.85.

- Still fractional. We need to do further branching.
  - $x_2 = 0$ (S3)
  - $x_2 = 1$ (S4)
- However, we obtained one important information: The optimal value of (S1) can't be better than 21.85.
- In fact, since the optimal value of (S1) must be an integer, therefore, it is at most 21
- This is a trick often used in bounding

Example Continued

Consider (S3), we solve the LP relaxation, and get the optimal solution is $x_1 = 1$, $x_2 = 0$, $x_3 = 1$, $x_4 = 1$. This is an integral solution with optimal value 18. Therefore, we are done with this branch

- A lower bound of 18 is obtained
- The optimal value of the original IP is at least 18
- If we solve a later LP relaxation and get the optimal value less than 18, then we don't need to further consider that branch

Consider (S4), we solve the LP relaxation, and get the optimal solution $x_1 = 0.6$, $x_2 = 1$, $x_3 = 1$, $x_4 = 0$ and the optimal value is 21.8.

- Still fractional. We need to do further branching.
  - $x_1 = 1$ (S5)
  - $x_1 = 0$ (S6)

Example Continued

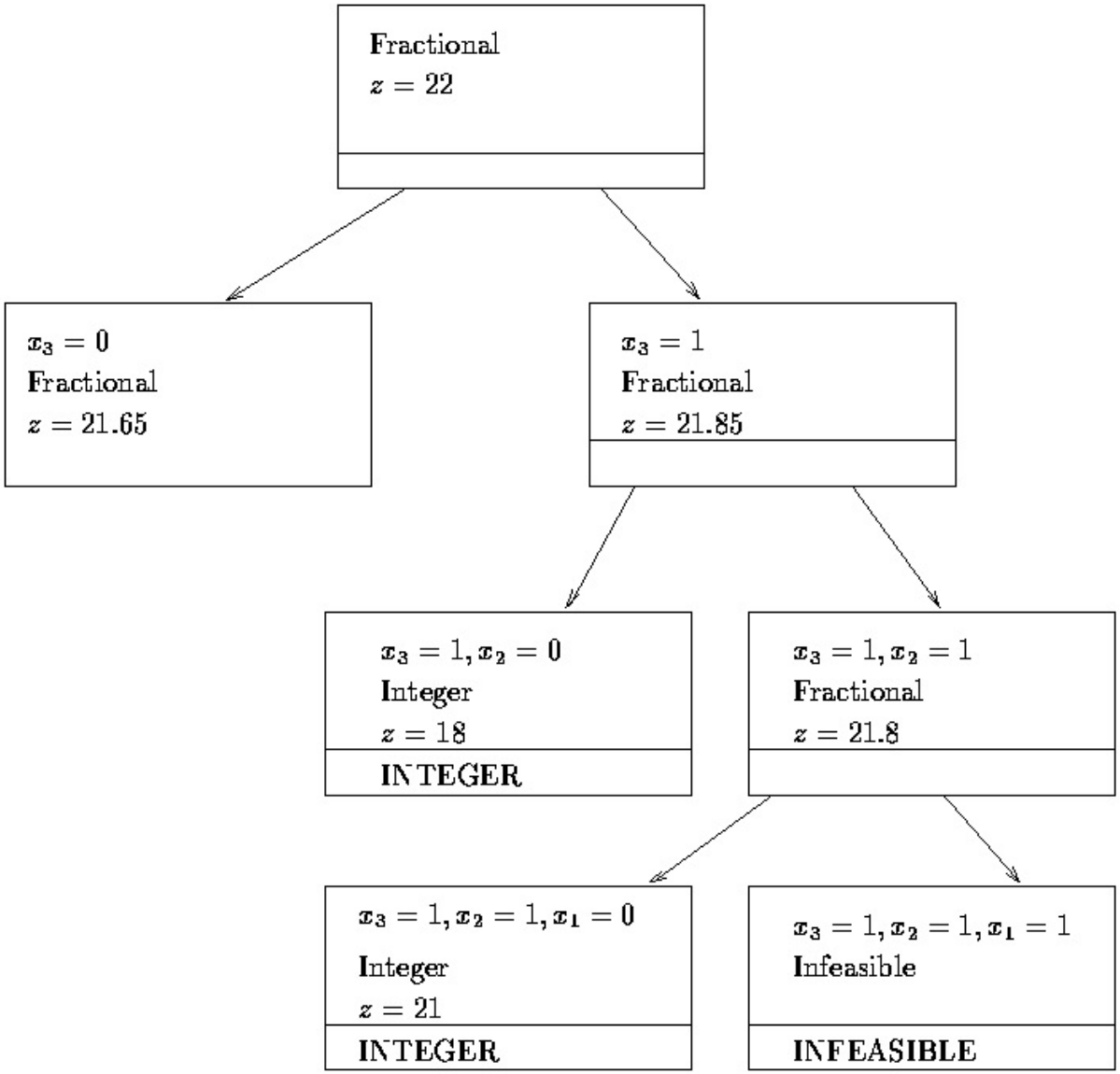
Consider (S5), it is easy to see that (S5) is infeasible. So we don't need to further consider it

Consider (S6), the optimal solution is $x_1 = 0$, $x_2 = 1$, $x_3 = 1$ and $x_4 = 1$. The optimal value is 21.

- Therefore, we get 21 as the optimal value for the first branch (S1) (optimal solution (0,1,1,1)).

Now consider (S2), we compute the LP relaxation and the optimal value is 21.65

- 21.65 is an upper bound on this branch
- Since the optimal value of (S2) must be integer, it means it can't be larger than 21.
- Therefore, no better solution can be obtained in this branch. We don't need to consider it


Fractional
$z = 22$
$x_3 = 0$
Fractional
$z = 21.65$
$x_3 = 1$
Fractional
$z = 21.85$
$x_3 = 1, x_2 = 0$
Integer
$z = 18$
INTEGER
$x_3 = 1, x_2 = 1$
Fractional
$z = 21.8$
$x_3 = 1, x_2 = 1, x_1 = 0$
Integer
$z = 21$
INTEGER
$x_3 = 1, x_2 = 1, x_1 = 1$
Infeasible
INFEASIBLE

Example Summary

There are 16 possible combinations in total, but we don't need to visit all of them

- Bounding is very important, it can greatly reduce the search space
- In the above example, we don't need to consider the $x_3 = 0$ branch because of bounding

Summary

We studied a most popular algorithm for solving integer programs - the branch-and-bound method

- Understand this method (both branching and bounding)
- Can use this method to solve some small-sized integer or binary programs

The current landscape of solving Mixed Integer Linear Programming: here